

Efficient Model Development through Fine-tuning Transfer

Pin-Jie Lin¹

pinjie@vt.edu

Rishab Balasubramanian¹

rishbb@vt.edu

Fengyuan Liu²

fy.liu@mail.utoronto.ca

Nikhil Kandpal²

nkandpa2@cs.toronto.edu

Tu Vu¹

tuvu@vt.edu

¹Virginia Tech ²University of Toronto & Vector Institute

Abstract

Modern LLMs struggle with efficient updates, as each new pretrained model version requires repeating expensive alignment processes. This challenge also applies to domain- or language-specific models, where fine-tuning on specialized data must be redone for every new base model release. In this paper, we explore the transfer of fine-tuning updates between model versions. Specifically, we derive the *diff vector* from one *source* model version, which represents the weight changes from fine-tuning, and apply it to the base model of a different *target* version. Through empirical evaluations on various open-weight model versions, we show that transferring diff vectors can significantly improve the target base model, often achieving performance comparable to its fine-tuned counterpart. For example, reusing the fine-tuning updates from Llama 3.0 8B leads to an absolute accuracy improvement of 10.7% on GPQA over the base Llama 3.1 8B without additional training, surpassing Llama 3.1 8B Instruct. In a multilingual model development setting, we show that this approach can significantly increase performance on target-language tasks without retraining, achieving an absolute improvement of 4.7% and 15.5% on Global MMLU for Malagasy and Turkish, respectively, compared to Llama 3.1 8B Instruct. Our controlled experiments reveal that fine-tuning transfer is most effective when the source and target models are linearly connected in the parameter space. Additionally, we demonstrate that fine-tuning transfer offers a stronger and more computationally efficient starting point for further fine-tuning. Finally, we propose an iterative *recycling-then-finetuning* approach for continuous model development, which improves both efficiency and effectiveness. Our findings suggest that fine-tuning transfer is a viable strategy to reduce training costs while maintaining model performance.

1 Introduction

Today’s LLMs are developed in two main stages: (1) *pretraining* on vast text corpora using self-supervised objectives such as next-word prediction, and (2) *post-training*, which involves additional alignment steps, such as supervised fine-tuning and reinforcement learning to ensure the model aligns with human preferences. While this approach creates powerful and versatile LLMs, it poses a significant challenge for model updates. Specifically, each new version of the pretrained model requires repeating the alignment process, which is costly, particularly for frequent updates. This issue is further compounded when developing domain- or language-specific models, as fine-tuning on specialized data must be redone for every new base model release.

In this paper, we explore the transfer of fine-tuning updates between different model versions to improve the current LLM development process. Specifically, we propose incorporating the *weight updates* from a *source*

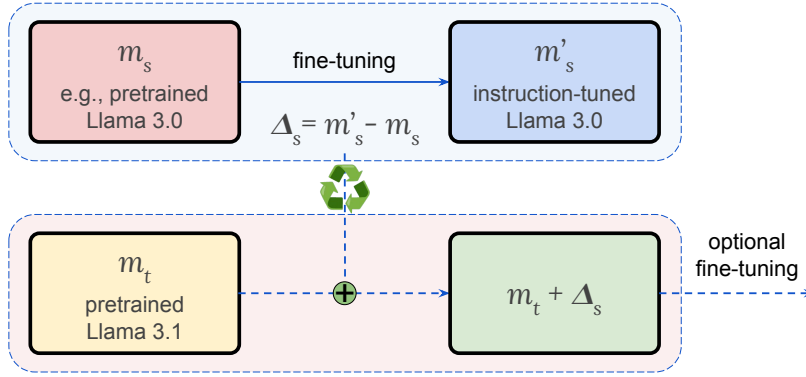


Figure 1: To transfer fine-tuning (e.g., instruction tuning) from a *source* model version s (e.g., Llama 3.0) to a *target* version t (Llama 3.1), we first compute the diff vector $\Delta_s = m'_s - m_s$ from version s , where m'_s is the fine-tuned model (instruction-tuned Llama 3.0) and m_s is the base model (pretrained Llama 3.0). Then, we add Δ_s to the target base model (pretrained Llama 3.1) to approximate the fine-tuned model in version t (instruction-tuned Llama 3.1). We explore two scenarios: (1) *recycling*—transferring from an older model version to a newer one to reduce retraining, and (2) *backporting*—transferring from a newer version to an older one to take advantage of the newer fine-tuning while maintaining optimization for specific use cases.

model version s to improve the training efficiency and effectiveness of a *target* model version t . Transferring from an older model version to a newer one (*recycling*) can save training time and computational resources by reducing the need for retraining. Conversely, transferring from a newer version to an older one (*backporting*) is beneficial when the older base model is better optimized for a specific use case (e.g., a particular language), allowing the user to take advantage of the new fine-tuning improvements while maintaining optimization and compatibility.¹

Our approach (see Figure 1) first computes the *diff vector* $\Delta_s = m'_s - m_s$ from version s , which represents the difference between the fine-tuned model m'_s (e.g., instruction-tuned) and its base model m_s (pretrained). Intuitively, Δ_s encodes the specific updates to the model parameters during fine-tuning, and can be used to transfer knowledge from the source version s (e.g., Llama 3.0) to the target version t (Llama 3.1). We hypothesize that models fine-tuned using the same or similar training data and procedures exhibit linear relationships across versions: $m'_s - m_s \approx m'_t - m_t$. This suggests that we can approximate the fine-tuned version m'_t of the target base model m_t without training: $m'_t \approx m_t + \Delta_s$. This is similar to the concept of *task vectors* (Ilharco et al., 2023), but instead of transferring across tasks using the same base model, we focus on transferring between different model versions trained on the same or similar data.

We begin by evaluating the feasibility of our approach through the transfer of diff vectors across different versions of open-weight models, including Llama (Dubey et al., 2024), OLMo (OLMo et al., 2024), and Tülu (Lambert et al., 2024). Our results show that fine-tuning updates can be successfully transferred across model versions. Specifically, by merging the diff vector Δ_s from a source model version s with the base model m_t of a target version t , we obtain a new model, $m_t + \Delta_s$, which significantly improves the performance of m_t on various tasks. In many cases, this method achieves performance comparable to the fine-tuned counterpart of m_t (i.e., m'_t) without requiring additional fine-tuning. For example, recycling the fine-tuning updates from Llama 3.0 yields a 10.7% absolute accuracy improvement on GPQA over the base Llama 3.1 8B, surpassing the performance of Llama 3.1 8B Instruct, without additional training.

Motivated by these results, we conduct a case study on the development of multilingual models. Specifically, we first fine-tune Llama 3.0 8B Instruct on target languages and then transfer the resulting diff vectors to Llama 3.1 8B Instruct. This recycling approach produces models that outperform Llama 3.1 Instruct in the target language without requiring additional training. For example, we obtain an absolute accuracy

¹In software development, *backporting* refers to the process of adapting features or updates from a newer version of a software system or component for use in an older version.

improvement of 4.7% for Malagasy and 15.5% for Turkish on the Global MMLU benchmark (Singh et al., 2024a), compared to Llama 3.1 8B Instruct.

To shed light on when fine-tuning transfer is most effective, we perform controlled experiments using OLMo 2’s intermediate pretrained checkpoints as different model versions. We fine-tune these models on the same data and investigate transferability across model versions. Our results suggest that fine-tuning transfer is most effective when the source and target models lie within a linearly connected region of the parameter space, indicating linear mode connectivity (Qin et al., 2023; Ainsworth et al., 2023; Wortsman et al., 2022a;b; Frankle et al., 2020).

Furthermore, we investigate whether the merged model $m_t + \Delta_s$ can serve as a computationally efficient and effective starting point for fine-tuning. Our experiments demonstrate that initializing fine-tuning with this merged model can accelerate convergence and improve accuracy compared to training on top of m_t . This suggests that fine-tuning transfer can be a beneficial intermediate step in scenarios where further training is feasible.

Finally, we explore a continuous model development scenario in which new model versions are regularly released. We propose an iterative *recycling-then-finetuning* approach that incrementally accumulates fine-tuning updates from previous model versions. Our experiments show that this method can consistently improve both training efficiency and model performance.

In summary, our key contributions are as follows.

- Introducing an approach for transferring fine-tuning updates between model versions via diff vector transfer.
- Demonstrating that this approach can reduce training costs while maintaining competitive performance.
- Validating the approach in a multilingual model development setting, showing improved language-specific performance without retraining.
- Establishing conditions for effective fine-tuning transfer, particularly when models exhibit linear mode connectivity.
- Proposing a recycling-then-finetuning strategy to improve both efficiency and performance in a continuous model development setting.

Terminology: In this paper, we use the terms “*transferring fine-tuning*”, “*transferring fine-tuning updates*” interchangeably to refer to the process of reusing fine-tuning updates (i.e., weight changes) from a source model version and incorporating them into a target version.

2 Transferring fine-tuning updates across model versions

In the development of modern LLMs, when a new pretrained model is released, fine-tuned models (optimized for specific tasks or languages) often need to be retrained to take advantage of the improvements, which can be computationally expensive. Recycling updates from an older model can mitigate this issue by reducing the need for full retraining. Conversely, when an older base model remains better suited for a specific use case, backporting updates from a newer model allows users to benefit from fine-tuning improvements while maintaining optimization and compatibility.

In this section, we explore transferring the weight changes from a source model version s to a target model version t , denoted $\mathcal{T}_{s \rightarrow t}$, without additional training. Specifically, we directly merge (add) the diff vector $\Delta_s = m'_s - m_s$ from version s , which captures the parameter adaptations from the base model m_s to its fine-tuned counterpart m'_s , onto the new base model m_t in version t , without any gradient-based training. Our results (Table 1) show that fine-tuning updates can be effectively transferred across model versions, as $m_t + \Delta_s$ often performs comparably to its fine-tuned counterpart m'_t .

Model	GSM8K	MATH	ARC _C	GPQA	MMLU	IFEval
Llama 3.0 8B Instruct	81.1	28.8	82.4	31.5	64.9	76.6
Llama 3.0 8B	55.6	17.3	79.7	22.3	66.7	34.5
+ $\Delta_{3.1}$	82.8	44.7	83.0	25.9	70.0	76.6
Llama 3.1 8B Instruct	86.5	50.3	83.8	31.3	72.9	80.5
Llama 3.1 8B	56.6	19.3	79.2	21.9	66.8	36.4
+ $\Delta_{3.0}$	79.8	29.9	82.9	32.6	65.1	83.3

Table 1: Fine-tuning transfer significantly improves the performance of the target base model across various tasks, achieving results comparable to its fine-tuned counterpart in many cases. Here, $\Delta_{3.0}$ and $\Delta_{3.1}$ represent the diff vectors between Llama Instruct and Llama for versions 3.0 and 3.1, respectively. **Notably, adding the diff vector Δ_s from a different model version can effectively transform a non-instruction-tuned model (e.g., Llama 3.0 or Llama 3.1) into one that follows instructions well (Llama 3.0 + $\Delta_{3.1}$ or Llama 3.1 + $\Delta_{3.0}$) without further training.** Additional results for OLMo and Tülu can be found in Appendix A, where we additionally find that advanced LLM capabilities, attained through alignment tuning stages such as Supervised Fine-Tuning (SFT), Direct Preference Optimization (DPO), or Group Relative Policy Optimization (GRPO), can be successfully transferred across different model versions.

2.1 Experimental setup

We conduct experiments with various open-weight models, including Llama (Dubey et al., 2024), OLMo (OLMo et al., 2024), and Tülu (Lambert et al., 2024). Our study explores both transfer directions: from an older model version to a newer one (*recycling*) and from a newer version to an older one (*backporting*). We emphasize that our goal is not to achieve state-of-the-art results, but instead to access the feasibility of transferring fine-tuning (by transferring weight changes) between model versions.

We evaluate the merged model $m_t + \Delta_s$ on a diverse set of benchmarks, including general knowledge with MMLU (Hendrycks et al., 2021a), math with GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021b), reasoning with ARC_C (Clark et al., 2018) and GPQA (Rein et al., 2024), and instruction-following with IFEval (Zhou et al., 2023). We compare its performance to that of fine-tuning m_t directly (i.e., m'_t).² See Appendix B for evaluation details.

2.2 Results and discussion

Transferring fine-tuning substantially boosts the target base model’s performance: Table 1 shows our results when transferring fine-tuning (i.e., instruction tuning) updates between Llama 3.0 and Llama 3.1. Strikingly, adding the diff vector Δ_s from a different model version can effectively transform a non-instruction-tuned model (e.g., Llama 3.0 or Llama 3.1) into one that follows instructions well (Llama 3.0 + $\Delta_{3.1}$ or Llama 3.1 + $\Delta_{3.0}$). For example, our approach yields 42.1% and 46.9% absolute accuracy improvements on the instruction-following benchmark IFEval over Llama 3.0 and Llama 3.1, respectively. Large gains are also observed across the board on math and reasoning benchmarks, including 27.2% over Llama 3.0 and 23.2% over Llama 3.1 on GSM8K. These results suggest that advanced knowledge and instruction-following abilities can be efficiently transferred between model versions without further training. In general, Llama 3.0 benefits more from the backported diff vector $\Delta_{3.1}$ from version 3.1 than Llama 3.1 does from recycling version 3.0’s diff vector $\Delta_{3.0}$.

Transferring fine-tuning can achieve performance comparable to the fine-tuned model: Our results demonstrate that the merged model $m_t + \Delta_s$ can perform on par with its fine-tuned counterpart m'_s across various tasks. This is particularly true for Llama 3.0 + $\Delta_{3.1}$, which matches or surpasses Llama 3.0 Instruct on five out of the six tasks we evaluated. Interestingly, Llama 3.1 + $\Delta_{3.0}$ outperforms Llama 3.1 Instruct on the GPQA and IFEval benchmarks. This is a testament to the diff vector’s ability to

²For evaluation, we use the `lm-evaluation-harness` library (Gao et al., 2024).

Model	Malagasy	Sinhala	Turkish
Llama 3.0 8B Instruct	23.1	23.3	30.8
+ FT	30.8	29.0	43.2
Llama 3.1 8B Instruct	27.6	33.0	27.7
+ $\Delta_{3.0}$	32.3	32.3	43.2

Table 2: Recycling fine-tuning updates improves multilingual performance on Global MMLU without re-training, yielding a 4.7% and 15.5% absolute improvement for Malagasy and Turkish, respectively, compared to Llama 3.1 8B Instruct. $\Delta_{3.0}$ represents the diff vector between Llama 3.0 Instruct and its monolingual fine-tuned (FT) version.

encode advanced reasoning and instruction-following capabilities. Overall, our results suggest that fine-tuning transfer provides an effective and extremely low-cost method to improve model performance when training is prohibitively expensive.

3 Efficient multilingual model development

Motivated by our results in Section 2, we now turn toward applying our fine-tuning transfer approach in a multilingual model development setting. We focus exclusively on a *recycling* scenario, where our aim is to transfer the language-specific instruction tuning updates from an older model version to a newer one. This approach helps avoid retraining, as frequently fine-tuning new model versions from scratch can be prohibitively expensive for multilingual communities.

For language-specific instruction tuning, we fine-tune an instruction-tuned model rather than a pretrained one. This approach aligns with the common practice of using an instruction-tuned English or multilingual model as the foundation when developing language-specific models. A key challenge in this setting is that state-of-the-art LLMs often include multilingual data in pretraining and instruction tuning, which makes it unclear whether language-specific fine-tuning is still necessary. *How effective is our recycling approach when applied to a multilingual instruction-tuned model?* Our results show that recycling fine-tuning remains effective in this scenario, as long as the base model is still outperformed by its previous monolingual counterpart. In such cases, our approach produces models that outperform Llama 3.1 Instruct on target-language tasks without additional training.

3.1 Experimental setup

We fine-tune Llama 3.0 Instruct (m_s) separately on language-specific instruction tuning data for three languages: Malagasy, Sinhala, and Turkish. We use the Aya dataset (Singh et al., 2024b) for Malagasy (14.6K examples) and Sinhala (14.5K examples), and the InstrucTurca dataset (Altinok, 2024) for Turkish (16.7K examples).³ Each model is trained for 30K training steps with a learning rate of 5e-6 and a batch size of 8, using 4 NVIDIA A100-80G GPUs.⁴

After training on each language, we compute the diff vector $\Delta_s = m'_s - m_s$ and add it to Llama 3.1 Instruct m_t . We focus on a low-resource setting, where no additional training on language-specific data is performed. Finally, we evaluate the merged model $m_t + \Delta_s$ against the base model m_t using the Global MMLU benchmark (Singh et al., 2024a).

	$\mathcal{M}_1$	$\mathcal{M}_2$	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	13.2	19.4	24.4	64.5	65.5
+ Δ_1		26.6	32.0	27.5	19.6
+ Δ_2	19.0		39.8	25.9	17.3
+ Δ_3	14.3	25.0		68.6	70.3
+ Δ_4	11.8	18.0	22.6		77.1
+ Δ_5	11.9	16.0	24.0	72.9	
FT($\mathcal{M}_i$)	45.1	50.7	60.4	75.7	75.5

Table 3: GSM8K accuracies indicating that more powerful models are better at leveraging transferred fine-tuning. Effective use of transferred fine-tuning only emerges once the target base model reaches a certain level of capability. Furthermore, fine-tuning transfer works best when the source and target models are close within a linearly connected region of the parameter space. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . FT($\mathcal{M}_i$) denotes applying fine-tuning directly to $\mathcal{M}_i$. See Table 11 in Appendix C for MATH500 results.

3.2 Results and discussion

Transferring fine-tuning is effective for developing multilingual models: Our results in Table 2 demonstrate the benefits of reusing fine-tuning updates in multilingual model development. For Malagasy and Turkish, transferring the diff vector from Llama version 3.0 to 3.1 results in significant accuracy improvements (4.7% and 15.5%, respectively) compared to Llama 3.1 8B Instruct. Our recycling approach also improves the fine-tuned Llama 3.0 Instruct model for Malagasy (1.5% accuracy improvement) without requiring additional training, while maintaining similar performance for Turkish. This is especially beneficial for multilingual communities, as it allows for model improvements at a very low cost by leveraging prior fine-tuning updates and an updated base model.

On the other hand, for Sinhala, recycling fine-tuning offers no advantage, as Llama 3.1 Instruct already outperforms the previously fine-tuned Llama 3.0 Instruct. However, even in this case, recycling does not significantly impact performance.

4 When is fine-tuning transfer effective?

Having demonstrated the effectiveness of fine-tuning transfer, we now conduct controlled experiments to better understand when this approach is most effective. At a high level, we treat different checkpoints of a pretrained model as distinct model versions. We then fine-tune these model versions on the same data and assess the impact of transferring fine-tuning updates between them. Our results reveal that fine-tuning transfer is most successful when the source and target models are close within a linearly connected region of the parameter space, indicating linear mode connectivity (Qin et al., 2023; Ainsworth et al., 2023; Wortsman et al., 2022a;b; Frankle et al., 2020).

4.1 Experimental setup

We conduct experiments with the publicly available intermediate pretrained checkpoints of OLMo 2 7B.⁵ The base OLMo 2 model was trained in two stages: (1) a general web-based pretraining stage (stage 1), and (2) a mid-training stage (stage 2) using high-quality web data and domain-specific data to enhance

³To simulate a low-resource setting, we sampled 6.5% of the original InstrucTurca dataset, which contains 2.58 million examples, resulting in approximately 16.7K examples.

⁴We use the AdamW optimizer with a linear scheduler and a warmup ratio of 0.03. We disable dropout and exclude weight decay for embeddings. The sequence length is 2048. We use open-instruct (Lambert et al., 2024) for training and lm-evaluation-harness (Gao et al., 2024) for evaluation.

⁵<https://huggingface.co/allenai/OLMo-2-1124-7B>

STEM-related capabilities. We select five checkpoints: $\mathcal{M}_1$ (early-stage 1, at 300K steps), $\mathcal{M}_2$ (mid-stage 1, at 600K steps), $\mathcal{M}_3$ (end-stage 1, at 929K steps), $\mathcal{M}_4$ (mid-stage 2, at 6K steps), and $\mathcal{M}_5$ (end-stage 2, at 12K steps). Each $\mathcal{M}_i$ is treated as a distinct model version. We investigate both transfer scenarios: (1) recycling from an older model version to a newer one ($\mathcal{T}_{\mathcal{M}_i \rightarrow \mathcal{M}_j}, i < j$), and (2) backporting from a newer version to an older one ($\mathcal{T}_{\mathcal{M}_j \rightarrow \mathcal{M}_i}, j > i$).

Due to our limited computational resources, supervised fine-tuning with a large instruction tuning dataset would be prohibitively expensive. We therefore fine-tune all model versions using a subset of the math reasoning instruction tuning data from Tülu 3, which includes Tülu 3 Persona MATH, GSM, and Algebra (220K examples total). We follow the training procedure described in Section 3.1.

We evaluate our models on GSM8K and the MATH500 subset (Hendrycks et al., 2021b) of the MATH dataset, which includes competition-level math problems of varying difficulty. These datasets are selected because fine-tuning on Tülu 3’s math reasoning data significantly improves performance on them, allowing for a clearer analysis of the impact of transferring fine-tuning updates between model versions.⁶

4.2 Results and discussion

More powerful models are better at leveraging transferred fine-tuning: Our results in Table 3 indicate that stronger models are more effective at leveraging transferred fine-tuning updates. While transferring fine-tuning can improve performance for $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$, the merged models $\mathcal{M}_i + \Delta_j$ (Δ_j denotes the diff vector from model version $\mathcal{M}_j$, $j \neq i$) still fall significantly short of their fine-tuned counterparts, denoted $\text{FT}(\mathcal{M}_i)$. On GSM8K, the accuracy gaps between the best $\mathcal{M}_i + \Delta_j$ and $\text{FT}(\mathcal{M}_i)$ are 26.1%, 24.1%, 20.6% for $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$, respectively. In contrast, for $\mathcal{M}_4$, this gap narrows to 2.8%. Notably, recycling fine-tuning from $\mathcal{M}_4$ to $\mathcal{M}_5$ (i.e., $\mathcal{M}_5 + \Delta_4$) surpasses fine-tuning directly on $\mathcal{M}_5$ ($\text{FT}(\mathcal{M}_5)$), achieving 1.6% accuracy improvement (77.1% vs. 75.5%). Similar trends are observed on MATH500. This pattern suggests an emergent ability—effective use of transferred fine-tuning only emerges when the target base model is sufficiently strong. In other words, the benefits of transferring fine-tuning only become significant beyond a certain level of capability.

Fine-tuning transfer works best when models are close in the parameter space: Our results also suggest that fine-tuning transfer is most effective when the source and target models are closely connected in the parameter space. On both GSM8K and MATH500, models $\mathcal{M}_1$ and $\mathcal{M}_2$ benefit more from Δ_3 than from Δ_4 or Δ_5 . Similarly, $\mathcal{M}_4$ and $\mathcal{M}_5$ gain more from Δ_3 than from Δ_1 or Δ_2 . Overall, $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$ form a mutually beneficial group, as do $\mathcal{M}_4$ and $\mathcal{M}_5$. However, transferring between these two groups can degrade performance. Specifically, $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$ do not benefit from Δ_4 and Δ_5 , while $\mathcal{M}_4$ and $\mathcal{M}_5$ typically benefit only from Δ_3 .⁷

5 Fine-tuning transfer as a starting point for further fine-tuning

So far, we have explored a scenario where fine-tuning is transferred between model versions without additional gradient-based training. We now switch gears to investigate whether the merged model $m_t + \Delta_s$ can serve as a stronger and more computationally efficient starting checkpoint for further fine-tuning. We conduct controlled experiments comparing two approaches: fine-tuning the merged model $m_t + \Delta_s$ versus fine-tuning m_t directly. Our results demonstrate that initializing fine-tuning with $m_t + \Delta_s$ often leads to faster convergence and higher performance on both seen and unseen tasks. This suggests that fine-tuning transfer can be a useful intermediate step when additional training is feasible. We refer to this approach as “*transferring-then-finetuning*”.

5.1 Experiment setup

We follow the training procedure outlined in Section 3.1. For evaluation, we use GSM8K and MATH500, along with several additional datasets to assess how well our transferring-then-finetuning approach general-

⁶For evaluation, we use the OLMES library (Gu et al., 2024).

⁷The only exception is $\mathcal{M}_4$ benefiting from $\mathcal{M}_1$ and $\mathcal{M}_2$ on MATH500.

	$\mathcal{M}_1$	$\mathcal{M}_2$	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	13.2	19.4	24.4	64.5	65.5
+ $\Delta_1 \rightarrow$ FT		56.9 _{+30.3}	62.8 _{+30.8}	77.8 _{+50.3}	78.6 _{+59.0}
+ $\Delta_2 \rightarrow$ FT	50.1 _{+31.1}		62.7 _{+22.9}	78.6 _{+52.7}	78.7 _{+61.4}
+ $\Delta_3 \rightarrow$ FT	48.5 _{+34.2}	57.6 _{+32.6}		77.6 _{+9.0}	78.8 _{+8.5}
+ $\Delta_4 \rightarrow$ FT	48.2 _{+36.4}	56.7 _{+38.7}	63.7 _{+41.1}		77.2 _{+0.1}
+ $\Delta_5 \rightarrow$ FT	48.1 _{+36.2}	55.6 _{+39.6}	63.5 _{+39.5}	76.2 _{+3.3}	
FT($\mathcal{M}_i$)	45.1	50.7	60.4	75.7	75.5

Table 4: **GSM8K accuracies showing that fine-tuning transfer provides a stronger starting point (i.e., $\mathcal{M}_i + \Delta_j$) for further fine-tuning (FT).** Numbers in subscript indicate improvement over the baseline without fine-tuning. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . FT($\mathcal{M}_i$) denotes applying fine-tuning directly to $\mathcal{M}_i$. See Table 12 in Appendix D for MATH500 results.

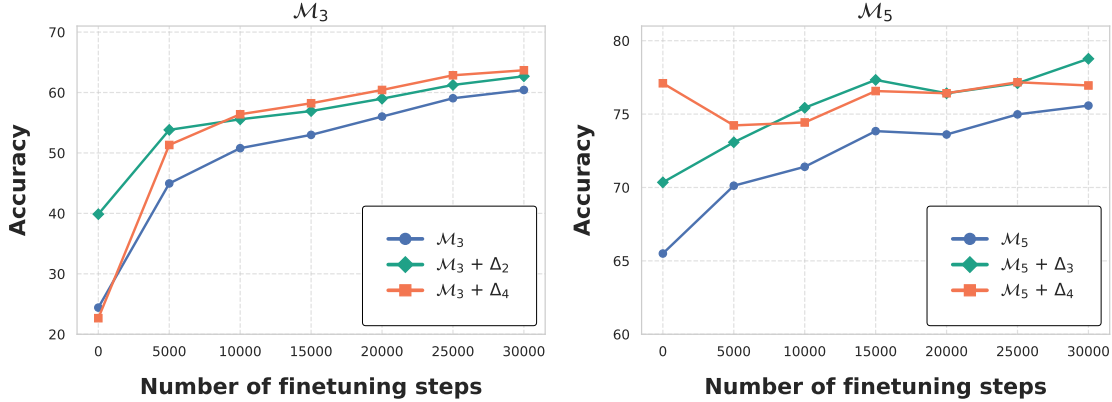


Figure 2: **GSM8K performance showing that fine-tuning transfer provides a more computationally efficient starting point (i.e., $\mathcal{M}_i + \Delta_j$) for further training.** Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . Additional results for $\mathcal{M}_1$, $\mathcal{M}_2$, $\mathcal{M}_4$ can be found in Appendix D.

izes to unseen tasks, including PhD-level science questions with GPQA_{Diamond} (Rein et al., 2024), tabular math word problems with TabMWP (Lu et al., 2023), and elementary school math word problems with ASDiv (Miao et al., 2020).

5.2 Results and discussion

Transferring-then-finetuning can substantially boost performance: Our results are summarized in Table 4. Transferring-then-finetuning offers significant improvements over our vanilla transfer approach (without additional fine-tuning) on both GSM8K and MATH500. On GSM8K, the largest accuracy improvements are 36.4%, 39.6%, 41.1%, 52.7%, and 61.4% for $\mathcal{M}_1$, $\mathcal{M}_2$, $\mathcal{M}_3$, $\mathcal{M}_4$, and $\mathcal{M}_5$, respectively. The benefits are most pronounced for weaker base models ($\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$) across all diff vectors, as well as for stronger base models when paired with a weak diff vector (e.g., $\mathcal{M}_5 + \Delta_1$).

Interestingly, fine-tuning also helps bridge the performance gap between the merged models $\mathcal{M}_i + \Delta_j$ ($j \neq i$) for each base model $\mathcal{M}_i$. For example, it dramatically improves the performance of $\mathcal{M}_5 + \Delta_1$ and $\mathcal{M}_5 + \Delta_2$ from 10.6% and 17.3% to 78.6% and 78.7%, respectively, closing the gap with the fine-tuned versions of $\mathcal{M}_5 + \Delta_3$ (78.8%) and $\mathcal{M}_5 + \Delta_4$ (77.2%). This reduces the need to pre-select the best diff vector when multiple

	GPQA _{Diamond}	TabMWP	ASDiv
$\mathcal{M}_5$	25.2	22.4	28.1
$+ \Delta_4 \rightarrow \text{FT}$	28.2	46.4	82.1
FT($\mathcal{M}_5$)	26.2	48.5	81.8

Table 5: **Transferring-then-finetuning does not negatively impact model generalization to unseen tasks.** Here, we recycle the diff vector Δ_4 from a previous pretrained version of OLMo 2 7B ($\mathcal{M}_4$) to a newer pretrained version ($\mathcal{M}_5$), before further fine-tuning the resulting model. Full GPQA_{Diamond} results can be found in Table 13 in Appendix D.

choices are available. Importantly, transferring-then-finetuning generally outperforms standard fine-tuning regardless of the diff vector used.

Transferring-then-finetuning can offer faster convergence: Figure 2 shows that using the merged model $\mathcal{M}_i + \Delta_j$ as the initial checkpoint improves training efficiency. Specifically, $\mathcal{M}_i + \Delta_j$ not only converges significantly faster than $\mathcal{M}_i$ during fine-tuning but also reaches a higher peak accuracy on GSM8K. Overall, our results suggest that transferring-then-finetuning is a cost-effective approach that reduces the number of fine-tuning steps, thereby improving training efficiency.

Transferring-then-finetuning does not negatively impact model generalization: As shown in Table 5, this approach attains strong zero-shot generalization on the three unseen tasks, comparable to standard fine-tuning. These results suggest that transferring-then-finetuning does not lead to overfitting, demonstrating its broad applicability across diverse tasks.

6 Iterative recycling-then-finetuning for improved performance and efficiency

Building on the insights from our previous experiments, we now explore a continuous model development setting in which new versions of a pretrained model are regularly released. At the core of our approach is an *iterative recycling-then-finetuning* strategy that incrementally incorporates fine-tuning updates, i.e., diff vectors, from past model versions. Instead of applying only the latest diff vector to the new base model, we recycle all previous diff vectors. Specifically, the diff vector at the current model version is carried forward to the next for subsequent fine-tuning. Our experiments show that this iterative recycling approach consistently improves both training efficiency and model performance.

6.1 Iterative recycling-then-finetuning

We treat the five intermediate checkpoints of OLMo 2 7B— $\mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3, \mathcal{M}_4, \mathcal{M}_5$ (described in Section 4.1) as different model versions of the pretrained OLMo 2 model. Our iterative recycling-then-finetuning algorithm, outlined in Algorithm 1, works as follows: At each iteration i , we first apply the most recent diff vector, Δ_{i-1}^{iter} , to the new base model $\mathcal{M}_i$, and then further fine-tune the resulting model. Next, we compute a new diff vector between the fine-tuned model and the current base model $\mathcal{M}_i$. This new diff vector is then carried forward to the next model version for fine-tuning in the subsequent iteration.

We refer to our iterative recycling-then-finetuning approach as Δ^{iter} and compare it to Δ^{dir} , a direct recycling-then-finetuning approach that applies the diff vector from the latest model version directly to the current model. We follow the training procedure outlined in Section 3.1.

6.2 Results and discussion

Iterative recycling-then-finetuning significantly improves performance: Table 6 compares the performance of two recycling approaches: iterative recycling-then-finetuning (Δ^{iter}) and direct recycling-then-finetuning (Δ^{dir}). Both approaches lead to significant performance improvements across model versions

Algorithm 1 Iterative recycling-then-finetuning

```
1: Notation: FT denotes fine-tuning
2: Input: Base models  $\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_n$ 
3: Output: Fine-tuned models  $\mathcal{M}_1^*, \mathcal{M}_2^*, \dots, \mathcal{M}_n^*$ 
4:  $\mathcal{M}_1^* \leftarrow \text{FT}(\mathcal{M}_1)$ 
5: for  $i = 2$  to  $n$  do
6:    $\Delta_{i-1}^{iter} = \mathcal{M}_{i-1}^* - M_{i-1}$ 
7:    $\mathcal{M}_i^* \leftarrow \text{FT}(M_i + \Delta_{i-1}^{iter})$ 
8: end for
9: return  $\mathcal{M}_1^*, \mathcal{M}_2^*, \dots, \mathcal{M}_n^*$ 
```

	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	24.4	64.5	65.5
+ $\Delta^{dir} \rightarrow \text{FT}$	62.7 _{+38.3}	77.6 _{+13.1}	77.2 _{+11.7}
+ $\Delta^{iter} \rightarrow \text{FT}$	67.0 _{+42.6}	77.3 _{+12.8}	77.5 _{+12.0}
FT($\mathcal{M}_i$)	60.4	75.7	75.6

Table 6: Both iterative (Δ^{iter}) and direct (Δ^{dir}) recycling-then-finetuning approaches significantly boost GSM8K performance, surpassing standard fine-tuning without recycling ($\text{FT}(\mathcal{M}_i)$). Numbers in subscripts indicate improvement over OLMo 2 7B checkpoints. At a high level, Δ^{iter} gradually incorporates fine-tuning updates, i.e., diff vectors, from previous model versions, while Δ^{dir} directly applies the diff vector from the latest model version to the current model. Results for $\mathcal{M}_1$ and $\mathcal{M}_2$ are omitted as these models remain identical across the two approaches (see Algorithm 1).

on GSM8K, with larger gains observed in earlier versions. For example, Δ^{iter} achieves absolute accuracy improvements of 42.6%, 12.8%, and 12% over $\mathcal{M}_3$, $\mathcal{M}_4$, and $\mathcal{M}_5$, respectively. Both approaches also outperform the standard fine-tuning baseline (without recycling) by a substantial margin. Specifically, Δ^{iter} yields accuracy improvements of 6.6% on $\mathcal{M}_3$ and 1.9% on $\mathcal{M}_5$ compared to standard fine-tuning. In general, Δ^{iter} performs similarly to or significantly better than Δ^{dir} across all model versions. These results suggest that in scenarios where the base model is continuously updated, adopting an iterative recycling strategy is beneficial.

Iterative recycling-then-finetuning leads to faster convergence: Figure 3 shows that both recycling approaches—iterative (Δ^{iter}) and direct (Δ^{dir}) recycling-then-finetuning—offer a more computationally efficient starting point for further fine-tuning. In general, Δ^{iter} consistently outperforms Δ^{dir} in terms of training efficiency and significantly improves standard fine-tuning without recycling. These results indicate that iterative recycling not only improves model performance but also enhances training efficiency by effectively leveraging the knowledge stored in the diff vectors across different model versions.

7 Related work

Fine-tuning transfer: We build on prior work in fine-tuning transfer and model merging. The literature on fine-tuning transfer is rife with methods that adapt the entire model (Phang et al., 2018; Pruksachatkun et al., 2020; Vu et al., 2020; 2021; Aghajanyan et al., 2021) or use parameter-efficient modules such as adapters (Pfeiffer et al., 2021; Poth et al., 2021), soft prompts (Vu et al., 2022b;a; Su et al., 2022; Asai et al., 2022), and LoRA matrices (Huang et al., 2024; Zadouri et al., 2024; Ostapenko et al., 2024); see Yadav et al. (2024a) for a comprehensive survey. However, most of these approaches focus on adapting the same base model to different tasks, domains, or languages. Similarly, model merging combines multiple task-specific models based on the same base model to create a more powerful model (Ilharco et al., 2023; Yadav et al., 2023; Yu et al., 2024; Yadav et al., 2024b; Ahmadian et al., 2024; Bandarkar et al., 2025). In contrast, our

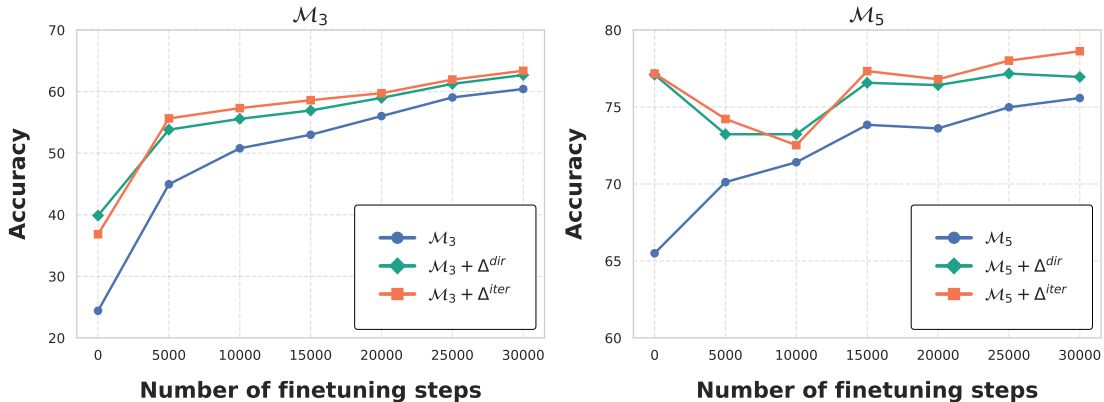


Figure 3: **GSM8K performance showing that both iterative (Δ^{iter}) and direct (Δ^{dir}) recycling-then-finetuning approaches offer faster convergence.** At a high level, Δ^{iter} gradually incorporates fine-tuning updates, i.e., diff vectors, from previous model versions, while Δ^{dir} directly applies the diff vector from the latest model version to the current model.

approach transfers fine-tuning updates across different model versions trained on the same or similar data, tackling the challenge of frequent model updates in modern LLM development.

Cross-model fine-tuning transfer: Previous studies (Lester et al., 2022; Su et al., 2022) investigate cross-model fine-tuning transfer, focusing mainly on transferring lightweight modules between non-instruction-tuned models. Lester et al. (2022) demonstrate that recycling soft prompts between models of the same size, as well as those of different sizes, is possible but challenging. Su et al. (2022) also investigate the transferability of soft prompts and show that they can be transferred between pretrained models of different architectures. In contrast, our work leverages diff vectors to enable transfer across model versions and explores both pretrained and instruction-tuned models.

Closely related to our work, Qin et al. (2023) study recyclable fine-tuning in a continual domain adaptation setting from the BERT (Devlin et al., 2019) era, where a pretrained RoBERTa (Liu et al., 2019) model is sequentially pretrained on new domains. They show that fine-tuning updates made to a domain-adapted checkpoint can be incorporated into a new one via element-wise multiplication, leading to improved performance and training efficiency on classification tasks. Their approach includes two recycling methods: initialization-based (similar to ours) and distillation-based (which minimizes KL divergence and mean-square loss between checkpoints). In this work, we explore a contemporary LLM development setting, focusing on today’s LLMs and open-ended generation benchmarks. We investigate the use of diff vectors to facilitate transfer across model versions in both recycling and backporting scenarios. Their approach and ours can complement each other, and combining them is a promising avenue for future work.

Finally, there is also a line of research focused on reusing small models for larger ones through methods such as duplication (Chen et al., 2022), progressive stacking (Gong et al., 2019), or merging parameters (Wang et al., 2023).

8 Conclusion

Our study demonstrates that fine-tuning transfer offers a practical approach to mitigate the inefficiencies of frequent model updates. By applying diff vectors from a fine-tuned source model version to a different target model version, we achieve substantial performance improvements without the need for full fine-tuning. In a multilingual context, this approach can significantly boost performance on target-language tasks, offering a scalable solution for language-specific fine-tuning without retraining on every new base model release. Through controlled experiments, we show that fine-tuning transfer is most effective when the source and target models are linearly connected in the parameter space. Furthermore, this approach can offer a more

robust and computationally efficient starting checkpoint for further fine-tuning and can be seamlessly integrated into a continuous model development cycle. Taken together, we hope that our work will spur more fundamental research into the efficient development of modern LLMs.

Acknowledgements

We thank Colin Raffel for valuable advice and useful suggestions on our experiments; Mohit Iyyer, Noah Constant, Tsendsuren Munkhdalai, Prateek Yadav, Naren Ramakrishnan, Alessandro Sordoni, Lucas Caccia, Minseon Kim, Ahmet Üstün, Tom Hosking, Matthias Gallé, Salaheddin Alzubi, Shayne Longpre, Quyet Do, Thinh Pham, Kavana Venkatesh, Nguyen Nguyen, Adam Nguyen, Rituraj Sharma, Anindita Chauhan, Yeana Lee, and the rest of the VT LLMs group for helpful discussions and suggestions at various stages in the project; and Brian Lester for sharing code for model merging.

References

- Armen Aghajanyan, Anchit Gupta, Akshat Shrivastava, Xilun Chen, Luke Zettlemoyer, and Sonal Gupta. Muppet: Massive multi-task representations with pre-finetuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5799–5811, 2021. URL <https://aclanthology.org/2021.emnlp-main.468/>.
- Arash Ahmadian, Seraphina Goldfarb-Tarrant, Beyza Ermis, Marzieh Fadaee, Sara Hooker, et al. Mix data or merge models? optimizing for diverse multi-task learning. *arXiv preprint arXiv:2410.10801*, 2024. URL <https://arxiv.org/abs/2410.10801>.
- Samuel Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=CQsmMYmlP5T>.
- Duygu Altinok. Instructurca: A diverse instructional content dataset for turkish, 2024. URL <https://huggingface.co/datasets/turkish-nlp-suite/InstrucTurca>.
- Akari Asai, Mohammadreza Salehi, Matthew Peters, and Hannaneh Hajishirzi. ATTEMPT: Parameter-efficient multi-task tuning via attentional mixtures of soft prompts. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 6655–6672, 2022. URL <https://aclanthology.org/2022.emnlp-main.446/>.
- Lucas Bandarkar, Benjamin Muller, Pritish Yuvraj, Rui Hou, Nayan Singhal, Hongjiang Lv, and Bing Liu. Layer swapping for zero-shot cross-lingual transfer in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=vQhn4wrQ6j>.
- Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. bert2BERT: Towards reusable pretrained language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2134–2148, 2022. URL <https://aclanthology.org/2022.acl-long.151/>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, 2019. URL <https://aclanthology.org/N19-1423/>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 3259–3269. PMLR, 2020. URL <https://proceedings.mlr.press/v119/frankle20a.html>.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2024. URL <https://zenodo.org/records/12608602>.

-
- Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tieyan Liu. Efficient training of BERT by progressively stacking. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2337–2346. PMLR, 2019. URL <https://proceedings.mlr.press/v97/gong19a.html>.
- Yuling Gu, Oyvind Tafjord, Bailey Kuehl, Dany Haddad, Jesse Dodge, and Hannaneh Hajishirzi. Olmes: A standard for language model evaluations. *arXiv preprint arXiv:2406.08446*, 2024. URL <https://arxiv.org/abs/2406.08446>.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a. URL <https://openreview.net/forum?id=d7KBjmI3GmQ>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021b. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. Lorahub: Efficient cross-task generalization via dynamic loRA composition. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=TrloAXEJ2B>.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=6t0Kwf8-jrj>.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024. URL <https://arxiv.org/abs/2411.15124>.
- Brian Lester, Joshua Yurtsever, Siamak Shakeri, and Noah Constant. Reducing retraining by recycling parameter-efficient prompts. *arXiv preprint arXiv:2208.05577*, 2022. URL <https://arxiv.org/abs/2208.05577>.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. URL <https://arxiv.org/abs/1907.11692>.
- Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=DHyHRBwJUTN>.
- Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing English math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 975–984, 2020. URL <https://aclanthology.org/2020.acl-main.92/>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024. URL <https://arxiv.org/abs/2501.00656>.
- Oleksiy Ostapenko, Zhan Su, Edoardo Maria Ponti, Laurent Charlin, Nicolas Le Roux, Matheus Pereira, Lucas Caccia, and Alessandro Sordani. Towards modular llms by building and reusing a library of lorae. *arXiv preprint arXiv:2405.11157*, 2024. URL <https://arxiv.org/abs/2405.11157>.

-
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. AdapterFusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 487–503, 2021. URL <https://aclanthology.org/2021.eacl-main.39/>.
- Jason Phang, Thibault Févry, and Samuel R Bowman. Sentence encoders on stilts: Supplementary training on intermediate labeled-data tasks. *arXiv preprint arXiv:1811.01088*, 2018. URL <https://arxiv.org/abs/1811.01088>.
- Clifton Poth, Jonas Pfeiffer, Andreas Rücklé, and Iryna Gurevych. What to pre-train on? Efficient intermediate task selection. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 10585–10605, 2021. URL <https://aclanthology.org/2021.emnlp-main.827/>.
- Yada Pruksachatkun, Jason Phang, Haokun Liu, Phu Mon Htut, Xiaoyi Zhang, Richard Yuanzhe Pang, Clara Vania, Katharina Kann, and Samuel R. Bowman. Intermediate-task transfer learning with pretrained language models: When and why does it work? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5231–5247, 2020. URL <https://aclanthology.org/2020.acl-main.467/>.
- Yujia Qin, Cheng Qian, Xu Han, Yankai Lin, Huadong Wang, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Recyclable tuning for continual pre-training. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 11403–11426, 2023. URL <https://aclanthology.org/2023.findings-acl.723/>.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pp. 53728–53741, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a85b405ed65c6477a4fe8302b5e06ce7-Paper-Conference.pdf.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Shivalika Singh, Angelika Romanou, Clémentine Fourrier, David I Adelani, Jian Gang Ngui, Daniel Vila-Suero, Peerat Limkonchotiwat, Kelly Marchisio, Wei Qi Leong, Yosephine Susanto, et al. Global mmlu: Understanding and addressing cultural and linguistic biases in multilingual evaluation. *arXiv preprint arXiv:2412.03304*, 2024a. URL <https://arxiv.org/abs/2412.03304>.
- Shivalika Singh, Freddie Vargus, Daniel D’souza, Börje Karlsson, Abinaya Mahendiran, Wei-Yin Ko, Herumb Shandilya, Jay Patel, Deividas Mataciunas, Laura O’Mahony, Mike Zhang, Ramith Hettiarachchi, Joseph Wilson, Marina Machado, Luisa Moura, Dominik Krzemiński, Hakimeh Fadaei, Irem Ergun, Ifeoma Okoh, Aisha Alaagib, Oshan Mudannayake, Zaid Alyafeai, Vu Chien, Sebastian Ruder, Surya Guthikonda, Emad Alghamdi, Sebastian Gehrmann, Niklas Muennighoff, Max Bartolo, Julia Kreutzer, Ahmet Üstün, Marzieh Fadaee, and Sara Hooker. Aya dataset: An open-access collection for multilingual instruction tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11521–11567, 2024b. URL <https://aclanthology.org/2024.acl-long.620/>.
- Yusheng Su, Xiaozhi Wang, Yujia Qin, Chi-Min Chan, Yankai Lin, Huadong Wang, Kaiyue Wen, Zhiyuan Liu, Peng Li, Juanzi Li, Lei Hou, Maosong Sun, and Jie Zhou. On transferability of prompt tuning for natural language processing. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3949–3969, 2022. URL <https://aclanthology.org/2022.naacl-main.290/>.

-
- Tu Vu, Tong Wang, Tsendsuren Munkhdalai, Alessandro Sordoni, Adam Trischler, Andrew Mattarella-Micke, Subhransu Maji, and Mohit Iyyer. Exploring and predicting transferability across NLP tasks. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7882–7926, 2020. URL <https://aclanthology.org/2020.emnlp-main.635/>.
- Tu Vu, Minh-Thang Luong, Quoc Le, Grady Simon, and Mohit Iyyer. STraTA: Self-training with task augmentation for better few-shot learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5715–5731, 2021. URL <https://aclanthology.org/2021.emnlp-main.462/>.
- Tu Vu, Aditya Barua, Brian Lester, Daniel Cer, Mohit Iyyer, and Noah Constant. Overcoming catastrophic forgetting in zero-shot cross-lingual generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 9279–9300, 2022a. URL <https://aclanthology.org/2022.emnlp-main.630/>.
- Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou’, and Daniel Cer. SPoT: Better frozen model adaptation through soft prompt transfer. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5039–5059, 2022b. URL <https://aclanthology.org/2022.acl-long.346/>.
- Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=cDYRS5iZ16f>.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23965–23998. PMLR, 2022a. URL <https://proceedings.mlr.press/v162/wortsman22a.html>.
- Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7959–7971, 2022b. URL https://openaccess.thecvf.com/content/CVPR2022/html/Wortsman_Robust_Fine-Tuning_of_Zero-Shot_Models_CVPR_2022_paper.html.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, volume 36, pp. 7093–7115, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1644c9af28ab7916874f6fd6228a9bcf-Paper-Conference.pdf.
- Prateek Yadav, Colin Raffel, Mohammed Muqeeth, Lucas Caccia, Haokun Liu, Tianlong Chen, Mohit Bansal, Leshem Choshen, and Alessandro Sordoni. A survey on model moerging: Recycling and routing among specialized experts for collaborative learning. *arXiv preprint arXiv:2408.07057*, 2024a. URL <https://arxiv.org/abs/2408.07057>.
- Prateek Yadav, Tu Vu, Jonathan Lai, Alexandra Chronopoulou, Manaal Faruqui, Mohit Bansal, and Tsendsuren Munkhdalai. What matters for model merging at scale? *arXiv preprint arXiv:2410.03617*, 2024b. URL <https://arxiv.org/abs/2410.03617>.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proceedings of the 41st International Conference on*

Machine Learning, volume 235 of *Proceedings of Machine Learning Research*, pp. 57755–57775. PMLR, 2024. URL <https://proceedings.mlr.press/v235/yu24p.html>.

Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermis, Acyr Locatelli, and Sara Hooker. Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=EvDeiLv7qc>.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL <https://arxiv.org/abs/2311.07911>.

Appendix

A Additional results for Section 2: Transferring fine-tuning updates across model versions

Model	GSM8K	MATH	ARC _C	GPQA	MMLU	IFEval
Llama 3.1 8B	56.6	19.3	79.2	21.9	66.8	36.4
Llama 3.1 8B Instruct	86.5	50.3	83.8	31.3	72.9	80.5
Tülu 3 8B SFT	76.2	31.6	79.1	31.0	65.1	72.0
Tülu 3 8B DPO	84.1	42.4	79.6	33.3	68.4	81.7
Tülu 3 8B	87.9	43.4	79.4	34.4	67.9	81.5
Llama 3.0 8B	55.6	17.3	79.7	22.3	66.7	34.5
+ Δ_{SFT}	71.8	26.3	77.9	32.1	63.5	69.1
+ Δ_{DPO}	81.1	38.1	78.6	31.9	67.5	82.9
+ Δ_{RLVR}	85.1	37.6	79.1	32.4	66.2	82.4
Tülu 3.1 8B	89.9	43.3	79.0	31.4	67.6	84.1
Llama 3.0 8B Instruct	81.1	28.8	82.4	31.5	64.9	76.6
Llama 3.0 8B	55.6	17.3	79.7	22.3	66.7	34.5
+ Δ_{GRPO}	85.8	39.5	78.2	29.4	65.0	82.6

Table 7: We find that advanced LLM capabilities, attained through alignment tuning stages such as SFT, DPO, RLVR, and GRPO (encoded in Δ_{SFT} , Δ_{DPO} , Δ_{RLVR} , and Δ_{GRPO} , respectively), can be successfully transferred across different model versions.

Model	GSM8K	MATH	ARC _C	GPQA	MMLU	IFEval
OLMo 2 7B	67.2	19.2	79.9	20.5	63.6	23.0
OLMo 2 7B SFT	71.7	25.2	79.7	27.9	61.2	67.7
OLMo 2 7B DPO	82.5	31.3	80.5	30.6	62.1	73.2
OLMo 2 7B Instruct	85.3	29.7	80.6	29.7	63.3	75.6
$\mathcal{M}_0$	2.5	1.6	25.7	18.1	25.0	12.2
+ Δ_{SFT}	2.2	0.8	23.8	1.3	1.4	13.7
+ Δ_{DPO}	2.1	0.8	24.1	1.1	1.3	13.7
+ Δ_{RLVR}	2.0	0.8	24.1	0.6	1.4	13.3
$\mathcal{M}_3$	24.4	5.7	72.7	15.4	59.8	15.7
+ Δ_{SFT}	31.7	8.4	74.3	24.8	55.4	51.4
+ Δ_{DPO}	40.4	9.3	75.0	29.9	56.6	68.0
+ Δ_{RLVR}	40.2	10.3	75.1	29.9	56.7	68.3
$\mathcal{M}_{4'}$	63.7	17.5	78.6	22.5	62.6	16.1
+ Δ_{SFT}	71.1	23.7	79.0	28.3	59.7	64.3
+ Δ_{DPO}	79.9	27.8	79.3	29.0	63.1	72.6
+ Δ_{RLVR}	82.8	27.7	79.3	27.2	62.2	72.1

Table 8: We find that advanced LLM capabilities, attained through alignment tuning stages such as SFT, DPO, and RLVR (encoded in Δ_{SFT} , Δ_{DPO} , and Δ_{RLVR} , respectively), can be successfully transferred across different model versions. Here, $\mathcal{M}_{4'}$ is an intermediate pretrained checkpoint of OLMo 2 7B (mid-stage 2, at 7K steps), which we selected before conducting our controlled experiments in Section 4.1.

We also conduct experiments with Tülu (Lambert et al., 2024) and OLMo (OLMo et al., 2024), both of which were developed from Llama 3.1 through multiple alignment stages, including Supervised Fine-Tuning (SFT), Direct Preference Optimization (DPO) (Rafailov et al., 2023), and a final reinforcement learning

stage—Reinforcement Learning with Verifiable Rewards (RLVR) (Lambert et al., 2024) for OLMo 2 and Tülu 3, or Group Relative Policy Optimization (GRPO) (Shao et al., 2024) for Tülu 3.1. At a high level, we subtract the weights of Llama 3.1 from these alignment-tuned checkpoints and then backport (add) the resulting diff vectors to Llama 3.0. Recycling is not applicable here, as we do not have the alignment-tuned checkpoints for Llama 3.0.

Our results are summarized in Table 7 and Table 8. In general, we find that advanced LLM capabilities, attained through alignment tuning stages such as SFT, DPO, RLVR, and GRPO (encoded in Δ_{SFT} , Δ_{DPO} , Δ_{RLVR} , and Δ_{GRPO} , respectively), can be successfully transferred across different model versions. For example, backporting Δ_{GRPO} from Tülu 3.1 8B to Llama 3.0 8B significantly improves accuracy, boosting GSM8K performance from 55.6% to 85.8% (30.2% improvement) and IFEval from 34.5% to 82.6% (48.1% improvement). This surpasses Llama 3.0 8B Instruct (81.1% on GSM8K, 76.6% on IFEval) and performs competitively with Llama 3.1 8B Instruct (86.5% and 80.5%) and Tülu 3.1 8B (89.9% and 84.1%).

B Additional evaluation details

Task	# Shots	CoT	Metric	Reference eval. setup
GSM8K	8	✓	exact match acc.	Llama 3 Evaluation Details ⁸
MATH	4	✓	exact match acc.	
ARC _C	0	✗	acc.	
GPQA	0	✓	exact match acc.	
MMLU	0	✓	exact match acc.	
IFEval	0	✗	avg. acc. (strict & loose)	Singh et al. (2024a)
Global MMLU	0	✗	acc.	

Table 9: Evaluation details for Llama 3.

Task	# Shots	CoT	Metric	Reference eval. setup
GSM8K	8	✓	exact match acc.	Lambert et al. (2024)
MATH	4	✓	flex exact match acc.	
ARC _C	5	✗	acc.	
GPQA	0	✓	exact match acc.	
MMLU	0	✓	exact match acc.	
IFEval	0	✗	prompt-level loose acc.	Muennighoff et al. (2025)
MATH500	0	✓	exact match acc.	
GPQA _{Diamond}	0	✓	exact match acc.	
TabMWP	0	✓	exact match acc.	Yang et al. (2024)
ASDiv	0	✓	exact match acc.	

Table 10: Evaluation details for OLMo 2 and Tülu 3.

We use the same evaluation setup and prompts as those in Llama 3 (Dubey et al., 2024) for Llama models and those in Tülu 3 (Lambert et al., 2024) for OLMo and Tülu models, whenever available. See Table 9 and Table 10 for more details. For evaluation, we use the `lm-evaluation-harness` library (Gao et al., 2024) for Llama models, and the `OLMES` library (Gu et al., 2024) for OLMo and Tülu models.

C Additional results for Section 4: When is fine-tuning transfer effective?

See Table 11.

⁸See https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/eval_details.md.

	$\mathcal{M}_1$	$\mathcal{M}_2$	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	14.6	11.6	11.4	11.6	16.6
+ Δ_1		8.8	17.8	19.2	15.6
+ Δ_2	7.6		12.6	14.6	14.4
+ Δ_3	8.0	9.4		23.4	27.8
+ Δ_4	7.8	8.0	9.8		34.2
+ Δ_5	8.0	7.4	11.2	30.6	
FT($\mathcal{M}_i$)	13.4	17.6	21.6	31.4	33.0

Table 11: MATH500 accuracies indicating that more powerful models are better at leveraging transferred fine-tuning. Effective use of transferred fine-tuning only emerges once the target base model reaches a certain level of capability. Furthermore, fine-tuning transfer works best when the source and target models are close within a linearly connected region of the parameter space. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . FT($\mathcal{M}_i$) denotes applying fine-tuning directly to $\mathcal{M}_i$.

D Additional results for Section 5: Fine-tuning transfer as a starting point for further fine-tuning

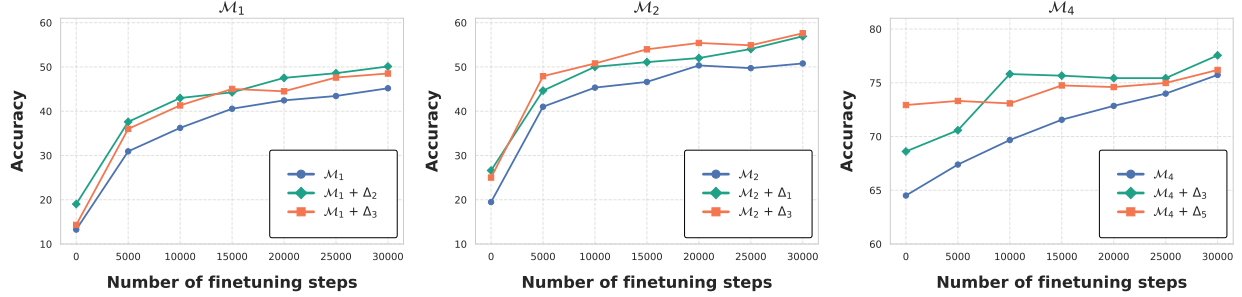


Figure 4: GSM8K performance showing that fine-tuning transfer provides a more computationally efficient starting point (i.e., $\mathcal{M}_i + \Delta_j$) for further training. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i .

See Table 12, Figure 4, and Table 13.

	$\mathcal{M}_1$	$\mathcal{M}_2$	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	14.6	11.6	11.4	11.6	16.6
+ $\Delta_1 \rightarrow$ FT		21.0 _{+12.2}	23.0 _{+5.2}	32.0 _{+12.8}	34.2 _{+18.6}
+ $\Delta_2 \rightarrow$ FT	16.2 _{+8.6}		26.2 _{+13.6}	31.6 _{+17.0}	31.0 _{+16.6}
+ $\Delta_3 \rightarrow$ FT	18.4 _{+10.4}	21.2 _{+11.8}		31.0 _{+7.6}	34.0 _{+6.2}
+ $\Delta_4 \rightarrow$ FT	17.4 _{+9.6}	19.0 _{+11.0}	23.8 _{+14.0}		36.2 _{+2.0}
+ $\Delta_5 \rightarrow$ FT	17.2 _{+9.2}	21.4 _{+14.0}	25.0 _{+13.8}	30.4 _{-0.2}	
FT($\mathcal{M}_i$)	13.4	17.6	21.6	31.4	33.0

Table 12: MATH500 accuracies showing that fine-tuning transfer provides a stronger starting point (i.e., $\mathcal{M}_i + \Delta_j$) for further fine-tuning (FT). Numbers in subscript indicate improvement over the baseline without fine-tuning. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . FT($\mathcal{M}_i$) denotes applying fine-tuning directly to $\mathcal{M}_i$.

	$\mathcal{M}_1$	$\mathcal{M}_2$	$\mathcal{M}_3$	$\mathcal{M}_4$	$\mathcal{M}_5$
	23.7	24.2	23.2	26.2	25.2
+ $\Delta_1 \rightarrow$ FT		25.2 _{+1.0}	25.1 _{+1.9}	33.3 _{+7.1}	25.7 _{+0.5}
+ $\Delta_2 \rightarrow$ FT	27.7 _{+4.0}		25.2 _{+2.0}	30.8 _{+4.6}	27.2 _{+2.0}
+ $\Delta_3 \rightarrow$ FT	27.7 _{+4.0}	27.7 _{+3.5}		23.7 _{-2.5}	23.2 _{-2.0}
+ $\Delta_4 \rightarrow$ FT	24.7 _{+1.0}	24.7 _{+0.5}	26.2 _{+3.0}		28.2 _{+3.0}
+ $\Delta_5 \rightarrow$ FT	26.7 _{+3.0}	26.7 _{+2.5}	23.2 _{+0.0}	25.7 _{-0.5}	
FT($\mathcal{M}_i$)	25.7	26.7	26.7	19.1	26.2

Table 13: GPQA_{Diamond} accuracies showing that fine-tuning transfer provides a stronger starting point (i.e., $\mathcal{M}_i + \Delta_j$) for further fine-tuning (FT), and transferring-then-finetuning does not negatively impact model generalization to unseen tasks. Numbers in subscript indicate improvement over the baseline without fine-tuning. Here, $\mathcal{M}_i$ represents different intermediate pretrained checkpoints of OLMo 2 7B (with smaller values of i indicating earlier checkpoints), and Δ_i refers to the diff vector resulting from the fine-tuning of version i . FT($\mathcal{M}_i$) denotes applying fine-tuning directly to $\mathcal{M}_i$.